Python for Data Science

Week(1): Introduction to Python & Programming Concepts

GitHub Repository

Moodle

Dr. Ghadeer Mobasher

Name Pronunciation

Ghadeer

IPA: /ʁa-deer/

Breakdown:

- **/ʁ/** : a voiced sound produced in the back of the throat, like the French *r* in *rouge*. which closely resembles the Arabic غ . It is **not** the English hard "g" as in "go ".
- **"a"** : like the "a" in *father* .
- **/deer/ :** stressed syllable, "deer" as in the animal, with a long /iː/ vowel.
- You may call me Dr. G ! ☺

About Me

- PhD in Computer Science, Heidelberg University.
- Research Associate at Heidelberg Institute for Theoretical Studies.
- My PhD focused on biomedical text and class imbalance problems.
- I enjoy teaching and find knowledge exchange genuinely rewarding.
- I believe no question is a bad question.
- I learn as much from failed experiments as from successful ones.
- I enjoy hiking and spending time in nature where I do my best thinking.



Rules & Values

- You are expected to **arrive on time** and **come prepared**, having completed any required pre-lab reading or setup.
- You should *engage actively* with the lab tasks, follow instructions carefully, and attempt all exercises independently **before seeking assistance**.
- Discussion and peer support are **encouraged**; however, each student must submit their own work.
- You should manage your **time effectively** to complete the assigned tasks within the allocated session.
- **All** practical lecture work must be submitted on **time** via your Moodle.
- Workstations should be kept tidy **before** you leave.
- You are expected to **maintain** a professional attitude, respect others, and minimise distractions during the session.



Module Overview

Examination Form

Group project with individual assessment

Teaching Methods

Problem-oriented learning, case studies, group work & individual exercises

Module Manger

Dr. Ghadeer Mobasher

Lecturers

Anil Keshav

Dr. Ghadeer Mobasher

Week	Topic
1	Introduction to Python & Programming Concepts
2	Control Flow & Logic
3	Core Data Structures
4	Functions & Modular Programming
5	Files & Input/Output
6	Data Analysis Foundations
7	Data Visualization, APIs & Clean Code

Anil's Lecture

Theoretical and technical concepts.

Ghadeer's Lecture

Hands-on exercises and submit them on Moodle at the end of the lecture.

Agenda

1. **Three quick anonymous polls**
   Getting to know you and breaking the 🧊
2. **“ Variable about me“ Card**
   Fill in the card and share the answer with your neighboring colleague.
3. **Why Python?**
   Opening Hook
4. **What is a Variable?**
   Variables & Data Types
5. **Arithemtic and Boolean Operations**
   Arithmetic & logical operators
6. **Why Type Conversion Matters?**
   Type Casting
7. **Putting it All Together**
   Exercises



What brought you to this programme? (Multiple Choice, select up to 2)



Rate your Python Experience



What do you want to build by the end of this module?

Take a screenshot and bring it back on the last day of the module to see how far you have come.

“ Variables about me “ card

```
# "Variable About Me" Card (5 min, individual, then share)
name          = "___"      # str
hometown      = "___"      # str
age           = ____       # int
fun_fact      = "___"      # str
dream_job     = "___"      # str
mood_today    = ____       # int (rate 1-5) 1 is grumpy and 5 is cheerful
attended_lecture= ____     # bool (TRUE/ False)
```

Tasks:

- Fill in the card within maximuim 5 minutes.
- Share the card with your neighboring colleague .

You may share it with two colleagues.

Why Python in Data Science & AI?

Rich Ecosystem
NumPy · Pandas · Matplotlib · Scikit-learn · TensorFlow, all in one language

Readable Syntax
Code reads almost like English. Less time decoding syntax, more time solving problems

Industry Standard
Used at Google, Meta, Nasa
#1 language in Kaggle surveys

Huge open-source community. A solution to almost any problem is a search way.

Core Data Types

A variable is a named label that points to a value stored in memory.

📎 Analogy: Think of a variable as a labelled box. The label is the name (e.g., age), the box holds the value (e.g. 69). You can re-label the box at any time.

```
# Declaring variables
first_name = 'Tom'      # str
age        = 69         # int
height     = 1.83       # float
is_married = False      # bool
skills     = ['Actor', 'Director']  # list
info       = {'city': 'Berlin'}     # dict
print(type(info))  # <class 'dict'>
```

Python is dynamically typed, no need to declare the type explicitly. The interpreter infers it.

Use **type()** to inspect any variable's type.

Core Data Types

Type	Description	Example	Result
int	Whole numbers including negative numbers	age = 69	69
float	Decimal numbers	height = 1.83	1.83
str	Text (in quotes)	name = 'Tom'	'Tom'
bool	True or False only	married = False	False
list	Ordered, mutable	nums = [1, 2, 3]	[1, 2, 3]
tuple	Ordered, immutable	films = ('A', 'B')	('A', 'B')
dict	Key → Value pairs	{'city': 'Berlin'}	{'city': 'Berlin'}



1. *What is the difference between a list and a tuple?*
2. *When would you use a dict instead of a list?*



In **Week1.ipynb** , navigate to the section " **Variables and Basic Data Types**" and experiment with the code in the **two** cells below.



In **Week1_Exercise.ipynb** , you have 7 minutes to solve "Problems 1 &2".



Week1_Exercise.ipynb
Download it first on your laptop

Core Data Types

f-strings are the recommended way to embed variables in strings.

Syntax: Prefix with f and wrap variables in { }.

```
# Old style – verbose
name = "Tom"
age  = 69
print("Hello, my name is " + name + " and I am
" + str(age))
```

```
# f-string – clean & modern
print(f"Hello, my name is {name} and I am {age}.")
print(f"Height: {1.83:.1f} metres")  # format float
print(f"2 + 2 = {2 + 2}")   # expressions work too!
```

▶ Output

```
Hello, my name is Tom and I am 69.
Height: 1.8 metres
2 + 2 = 4
```

Rule of thumb: use f-string whenever you need to mix variables and text.

Essential String Operations

A string is a **sequence of characters**.

Assume we have a language string variable as language='Python', it is equivalent to ['P ', 'y ', 't ', 'h ', 'o ', 'n '].

Operation	Syntax / Example	Description	Output Example
Length	len(language)	returns the number of characters	6
Indexing (first)	language[0]	accesses the first character	'P'
Indexing (last)	language[-1]	accesses the last character	'n'
Slicing (range)	language[0:3]	extracts characters from index 0 to 2	'Pyt'
Slicing (end)	language[-3:]	extracts the last three characters	'hon'
Slicing (step)	language[::2]	selects every second character	'Pto'

Essential String Operations (Cont.)

Operation	Syntax / Example	Description	Output Example
Concatenation	first_name + ' ' + last_name	combines strings with explicit separator	'John Doe'
Remove whitespace	sentence.strip ()	removes leading and trailing spaces	'hello, world!'
Uppercase	sentence.strip ().upper()	converts all characters to uppercase	'HELLO, WORLD!'
Title case	sentence.strip ().title()	capitalises each word	'Hello, World!'
Replace substring	replace(' world','Python ')	replaces part of a string	'hello, Python!'
Split string	'hello world'.split ()	splits into a list of words	['hello', 'world']



In **Week1.ipynb** , navigate to the section " **Modern String Formatting: f -strings** " & "**Essential String Operations** " and experiment with the code in the cells below.

Arithematic Operators

Operator	Name	Example (a=7,b=2)	Result
+	Addition	7 + 2	9
-	Subtraction	7 - 2	5
*	Multiplication	7 * 2	14
/	True Division	7 / 2	3.5 ← always float!
//	Floor Division	7 // 2	3 ← drops decimal
%	Modulus	7 % 2	1 ← remainder only
**	Exponentiation	7 ** 2	49

```
a, b = 7, 2
print(f"{a} / {b}  = {a / b}")    # 3.5
print(f"{a} // {b} = {a // b}")   # 3
print(f"{a} % {b}  = {a % b}")    # 1
```

```
▶ Output
7 / 2  = 3.5
7 // 2 = 3
7 % 2  = 1
```



How does the modulo operator work?

Understanding Modulo (%)

Modulo gives you the REMAINDER **after** integer division.

7 % 3 : Step by Step

Step 1: $7 \div 3 = 2$ (with a leftover)
Step 2: $3 \times 2 = 6$
Step 3: $7 - 6 = 1$ ← this is 7 % 3

In other words, 3 fits into 7 twice because 3*2=6
The remainder is 7-6=1
Thus, 7%3=1



In **Week1.ipynb** , navigate to the section "**Arithmetic Operations**" and experiment with the code in the cells below.



In **Week1_Exercise.ipynb** , you have 15 minutes to solve "Problems 3&4".

Boolean Comparisons

Operator	Meaning	Example	Result
==	Equal to	3 == 3	True
!=	Not equal	3 != 2	True
>	Greater than	5 > 3	True
<	Less than	2 < 1	False
>=	Greater or equal	5 >= 5	True
<=	Less or equal	4 <= 3	False

```
x, y = 10, 15
print(f"x > y  → {x > y}")
print(f"x == 10 → {x == 10}")
```

```
▶ Output
x > y  → False
x == 10 → True
```



What is the difference between a=15 and a==15?

Logical Operators

Operator	Rule	Example	Result
and	Both must be True	True and False	False
or	At least one must be True	True or False	True
not	Flips the value	not True	False

```
a, b = 8, 12

print(a < 10 and b > 10)  # True and True  → True
print(a > 5  or  b < 10)  # True or False  → True
print(not (a == 8))       # not True       → False

# Chained comparison (Pythonic!)
num = 7
print(5 < num < 10)       # equivalent to: 5 < num and num < 10
```

```
▶ Output
True
True
False

True
```



Simple Tracing: what does not (5 < 7 and 7 < 10) evaluate to? How about not (5 < 7) or 7 < 10 ?



In **Week1.ipynb** , navigate to the section "**Boolean Comparisons & Logical Operators** " and experiment with the code in the cells below.



In **Week1_Exercise.ipynb** , you have 15 minutes to solve "Problems 5&6".

Pause / Break

See you in 15 minutes ☺!
Please be on time..

Why Type Casting matters?

Data from files, APIs, or user input almost **always arrives as strings** even when it's a number. You must cast it before computing.

```
# Simulating data from a CSV file
user_age = "25"    # this came in as a string

# This crashes!
# result = user_age + 5
# TypeError: can only concatenate str (not "int") to str

# Fix: cast first
user_age_int = int(user_age)
result = user_age_int + 5
print(f"Age in 5 years: {result}")
```

```
▶ Output
Age in 5 years: 30
```

Function	Converts to	Example	Result
int()	Integer	int('20')	20
float()	Float	float('3.5')	3.5
str()	String	str(42)	'42'
bool()	Boolean	bool(0)	False

Truthiness – bool() on Anything

Python can evaluate any value as True or False. This is called truthiness.

❌ **Falsy values**

```
#0  ·  0.0  ·  ""  ·  []  ·  {}  ·  None

bool(0)   → False
bool("")  → False
bool([])  → False
```

✅ **Truthy values**

```
Everything else!

bool(1)       → True
bool(-5)      → True
bool("hi")    → True
bool("  ")    → True
bool([1,2,3]) → True
```

Surprise: bool(" ") → True! A string with only a space is NOT empty, it has content. This matters when validating user input.

Int() Truncates and Variable Reassignment

int() truncates does not round

```
# int() TRUNCATES, it does NOT round!
print(int(44.4))    # 44
print(int(44.9))    # 44  ← not 45!
print(round(44.9))  # 45  ← use round() to round
```

```
▶ Output
44
44
45
```

Variables are just labels that can be re-pointed

```
# Variable reassignment — labels vs objects
x = 5
print(f"x = {x}, id = {id(x)}")
x = 10
print(f"x = {x}, id = {id(x)}")
# id() changed → x now points to a NEW object
# The integer 5 itself was never modified
```

```
▶ Output

x = 5,  id = 140234...
x = 10, id = 140235...
(different memory address)
```



1. Why does Python say integers are 'immutable' if you can clearly reassign *x*?
2. What exactly is immutable, the variable or the object?



In **Week1.ipynb** , navigate to the section "**Data Type Conversions (Type Casting)** " and experiment with the code in the cells below.



In **Week1_Exercise.ipynb** , you have 15 minutes to solve "Problems 7-11".

Operator Precedence (PEMDAS)

Priority	Rule	Operator(s)
1st	Parentheses	( )
2nd	Exponentiation	**
3rd	Multiplication / Division	* / // %
4th	Addition / Subtraction	+ -

```
# Let's evaluate together — predict before running!
# (5 + 3) * 2**3 - 4/2

# Step 1 — Parentheses:    (5+3) = 8
# Step 2 — Exponent:       2**3  = 8
# Step 3 — Multiply:       8 * 8 = 64
# Step 4 — Divide:         4/2   = 2.0
# Step 5 — Subtract:       64 - 2.0 = 62.0

result = (5 + 3) * 2**3 - 4/2
print(result)   # 62.0
```

```
▶ Output
62.0
```

In **Week1_Exercise.ipynb** , you have 7 minutes to solve "Problem 12".

Integration Challenge: Banking Transaction

Problem (13) in **Week1_Exercise.ipynb** combines:

- Variables
- Type casting
- Boolean Comparison
- Arithmetic Operations
- Print Formatting



In **Week1_Exercise.ipynb** , you have 7 minutes to solve "Problem 13".



What if withdrawal_amount was '1500'? Which variables change, and which don't? What would can_withdraw be?

Additional Exercises

P14

Student Enrollment

Combine age (string → int), credits (int), and two booleans to determine if a student can enroll.

Skills tested: Type casting · Boolean logic · Variables

P15

Mixed Calculation Challenge

Cast a='14' and c='3' to numeric, then compute result1=(a+b)*c, result2=(b**c)/a, result3=(a*c)%b.

Skills tested: All arithmetic operators · Type casting · Predicting output types

What did we learn today?

Environment

Jupyter Notebook, integrated within Visual Studio Code, serves as the primary coding environment

Variables

Named labels pointing to typed values in memory

Data Types

int · float · str · bool · list · tuple · dict

f-strings

Modern, clean string formatting with embedded expressions

Arithmetic

7 operators: + − * / // % ** : know when / returns float

Booleans

Comparisons (== != > < >= <=) and logic (and or not)

Type Casting

int() float() str() bool() : essential for data cleaning

Immutability

Variables are labels; integers are immutable objects

Coming Up : Week (2)

What if your code makes decisions?

- if / elif / else
- for loops
- while loops
- Functions

Any Questions?

See you next time ☺

Dr. Ghadeer Mobasher
ghadeer.mobasher@srh-hochschulen.de
https://www.linkedin.com/in/ghadeermobasher/
LGS 6, 2. OG, Arc 213